Global vs. local variables

```
#include <stdio.h>
#define PI 3.14

int count;

int func()
{
   int i, j;

   scanf("%d %d", &i, &j);
   count += i+j;
   return 0;
}

int main()
{
   int i;

   func();
   for (i=0; i<count; i++)
      printf("%d",i);
   return 0;
}
```

Global variable

Local variables of func()

Local variable of main()

Scope of variables

- A local variable can be used only in the function where it is defined
  - i.e., the scope of a local variable is the current function.
- A global variable can be used in all functions below its definition
  - i.e., the scope of a global variable is the range from the variable definition to the end of the file.

Lifetime of variables

- The lifetime of a variable depends on its scope.
- A local variable is alive as long as the function where it is defined is active.
  - When the function terminates, all of the local variables (and their values) are lost.
  - When the function is called again, all variables start from scratch; they don't continue with their values from the previous call (except for local static variables which are not discussed).
- The lifetime of a global variable is equivalent to the lifetime of the program.

Scope and lifetime of variables

```
#include <stdio.h>
#define PI 3.14

int count;

int func()
{
   int i, j;

   scanf("%d %d", &i, &j);
   count += i+j;
   return 0;
}

int main()
{
   int i;

   func();
   for (i=0; i<count; i++)
      printf("%d",i);
   return 0;
}
```

scope of count

scope of i,j

scope of i

"count" is visible in all functions

Global vs. local variables

Global variables	Local variables
Visible in all functions	Visible only within the function they are defined
Zero by default	Uninitialized
A change made by a function is visible everywhere in the program	Any changes made on the value are lost when the function terminates (since the variable is also removed)
Scope extends till the end of the file	Scope covers only the function
Lifetime spans the lifetime of the program	Lifetime ends with the function

Changing local variables

- Any change made on local variables is lost after the function terminates.

```
void f()
{   int a=10;
    a++;
    printf("in f(): a=%d\n",a);
}
int main()
{   int a=5;
    f();
    printf("After first call to f(): a=%d\n",a);
    f();
    printf("After second call to f(): a=%d\n",a);
}
```

Changing global variables

- Any change made on global variables remains after the function terminates.

```
int b;
void f()
{
   b++;
   printf("in f(): b=%d\n",b);
}
int main()
{
   f();
   printf("After first call to f(): b=%d\n",b);
   f();
   printf("After second call to f(): b=%d\n",b);
}
```

This is called side effect. Avoid side effects.

Changing value parameters

- Any change made on value parameters is lost after the function terminates.

```
void f(int c)
{
    c++;
    printf("in f(): c=%d\n",c);
}
int main()
{   int c=5;
    f(c);
    printf("After f(): c=%d\n",c);
}
```

Local definition veils global definition

- A local variable with the same name as the global variable veils the global definition.

```
int d=10;
void f()
{
   d++;
   printf("in f(): d=%d\n",d);
}
int main()
{  int d=30;
   f();
   printf("After first call to f(): d=%d\n",d);
   f();
   printf("After second call to f(): d=%d\n",d);
}
```

Parameter definition veils global definition

- Similar to local variables, parameter definition with the same name as the global variable also veils the global definition.

```
int e=10;
void f(int e)
{
   e++;
   printf("in f(): e=%d\n",e);
}
int main()
{  int g=30;
   f(g);
   printf("After first call to f(): g=%d\n",g);
}
```

Example

- Write a function that calculates the factorial of its parameter.

$$n! = n(n-1)(n-2)...1$$

```
long factorial(int n)
{  int i; long res = 1;
   if (n == 0)
      return 1;
   for (i = 1; i <= n; i++)
      res *= i;
   return res;
}
```

Example

- Write a function that calculates the permutation

  $P(n, k) = n!/(n-k)!$

```
long perm(int n, int k)
{  long result;
   if ((n<0)||(k<0)||(n<k))
      return 0;
   else
   {  result = factorial(n)/factorial(n-k);
      return result;
   }
}
```

Example

- Write a function that calculates the combination

$$C(n, k) = n!/[(n - k)! \cdot k!]$$

```
long comb(int n, int k)
{  long result;
   if (n < 0 || k < 0 || n < k)
      return 0;
   else
      return(factorial(n)/(factorial(n-k)*factorial(k)));
}
```

Macro Substitution

- Remember

```
#define PI 3.14
```

- It is also possible to write macros with parameters.

```
#define square(x) x*x
```

  - The name of the macro is **`square`**.
  - Its parameter is **`x`**.
  - It is replaced with **`x*x`** (with paramater **`x`** substituted.)

Macro Substitution

- Note that when the macro is called as

```
square(a+1)
```

  the substituted form will be

```
a+1*a+1
```

  which is not correct.

- The macro should have been defined as

```
#define square(x)  (x)*(x)
```

  so that its substituted form would be

```
(a+1)*(a+1)
```

Macro Substitution

- A macro is NOT a function.
  - A macro is implemented as a substitution.
    - The code segment that implements the macro substitutes the macro call.
    - \+ Code executes faster.
    - \- Code becomes larger.
    - \+ It is possible to do things that you cannot do with functions.
    - \- Limited syntax check (Remember the `"square(a+1)"` example).

Macro Substitution

- A function is implemented by performing a jump to a code segment.
  - Stack operations are performed for function call.
  - \- Code executes slower.
  - \- Code is smaller.
  - \+ More structured programming.
  - \+ Better syntax check.

Example: Macro substitution

- Define a macro for finding the maximum of two values.

```
#define max(A,B) (((A)>(B))?(A):(B))
```

Example: Macro substitution

- Define a macro for finding the absolute value.

```
#define abs(A)  (((A)>0)?(A):-(A))
```

Example

- Write a function that exchanges its parameters.
- Solution 1:

```
void swap(int a, int b)
{
    a=b;
    b=a;
}
```

WRONG!

Example

- Solution 2:

```
void swap(int *a, int *b)
{
    *a=*b;
    *b=*a;
}
```

STILL WRONG!

Example

- Solution 3:

```
void swap(int *a, int *b)
{
    int temp;
    temp = *a;
    *a=*b;
    *b=temp;
}
```

Example

- What is the output of this one?

```
#include <stdio.h>

int f(int x, int y)
{
    printf("in f(): x=%d   y=%d \n", x, y);
    return x+y;
}
int g(int x, int y)
{
    printf("in g(): x=%d   y=%d \n", x, y);
    return y-x;
}
int h(int x, int y)
{
    printf("in h(): x=%d   y=%d \n", x, y);
    return x/y;
}

int main()
{
    int a=10, b=20, c=30, d=40;
    printf("in main(): %d\n", h(f(a,b),g(c,d)));
    return 0;
}
```

Recursive functions

- Just a simple example of recursive functions.

```
#include <stdio.h>
long factorial(int n)
{
   if (n>0)
     return n*factorial(n-1);
   else if (n==0)
     return 1;
   return 0;
}

int main()
{
   int n;
   printf("Enter an integer: ");
   scanf("%d", &n);
   printf("%d! = %ld\n", n, factorial(n));
   return 0;
}
```

- As we said before, avoid recursive functions.